

ĐẠI HỌC QUỐC GIA TP.HCM
TRƯỜNG ĐẠI HỌC BÁCH KHOA



KIẾN TRÚC MÁY TÍNH
(CO2008)

BÁO CÁO BÀI TẬP LỚN
ĐỀ 6
CHƯƠNG TRÌNH IN PHỔ

GVHD: NGUYỄN XUÂN MINH
Nhóm 24

STT	Họ và tên	MSSV	Nội dung thực hiện	Hoàn thành (%)
1	Nguyễn Hữu Tây	2413095	Tìm hiểu thuật toán, soạn báo cáo latex	100
2	Lê Quốc Linh	2411875	Hiện thực bài toán	100

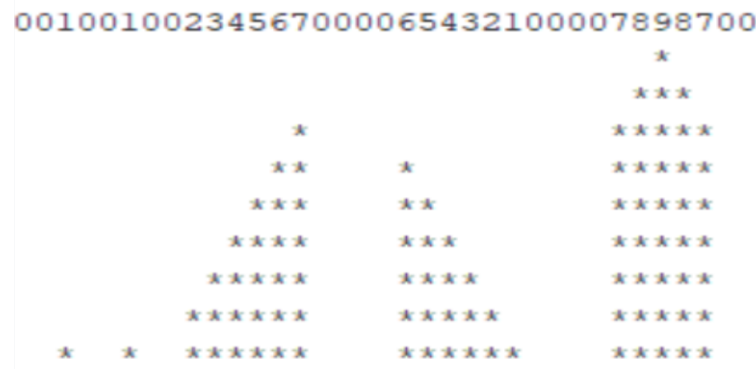
TP.HCM, 11/2025

Mục lục

1	Giới thiệu đề bài	2
2	Giải pháp hiện thực	2
2.1	Data Segment: Định nghĩa dữ liệu cho chương trình	2
2.2	Code Segment	2
2.2.1	Bước 1: Lấy dữ liệu đọc được từ tập tin STRING.TXT và kiểm tra tính hợp lệ . .	2
2.2.2	Bước 2: Chuẩn bị dữ liệu và tìm chiều cao biểu đồ	3
2.2.3	Bước 3: In ra phổ thỏa mã với yêu cầu	3
2.2.4	Bước 4: Kết thúc chương trình	3
3	Thống kê số lệnh, loại lệnh (instruction type) sử dụng trong chương trình	3
3.1	Số lệnh	3
3.2	Các loại lệnh dùng trong chương trình	4
4	Tính thời gian chạy của chương trình	6
4.1	Xác định chu kỳ CR	6
4.2	Xác định IC	6
4.2.1	Tính IC ở bước 1	6
4.2.2	Tính IC ở bước 2	7
4.2.3	Tính IC ở bước 3	7
4.2.4	Tính IC ở bước 4	7
4.2.5	Tổng IC của chương trình	7
4.3	Tính tổng thời gian chạy	7
5	Kết quả kiểm thử	7
5.1	Test 01	7
5.2	Test 02	8
5.3	Test 03	8
6	Lời cảm ơn	9

1 Giới thiệu đề bài

Viết chương trình in phổ sao như hình bên dưới:



Chuỗi ≤ 40 ký số ASCII đọc từ tập tin STRING.TXT trên đĩa.
 Kết quả này cũng phải được lưu lên tập tin PHO_SAO.TXT trên đĩa.

2 Giải pháp hiện thực

Phần hiện thực chương trình của nhóm chúng em sẽ bao gồm:

Bước 1: Lấy mảng tối đa 40 số đọc được từ tập tin STRING.TXT.

Bước 2: Kiểm tra tính hợp lệ và in ra chuỗi các ký số đọc được.

Bước 3: Vẽ biểu đồ phổ sao (phổ) theo chiều dọc dựa trên các giá trị trong chuỗi.

Bước 4: Ghi kết quả vào tập tin PHO_SAO.TXT và in ra console.

2.1 Data Segment: Định nghĩa dữ liệu cho chương trình

- Vùng đọc dữ liệu từ file: Theo yêu cầu đề bài, nhóm định nghĩa các biến chứa đường dẫn của file "STRING.TXT" chứa chuỗi số đầu vào và "PHO_SAO.TXT" để ghi kết quả phổ sao được vẽ ra theo yêu cầu.
- Vùng đệm không gian buffer_doc (tối đa 40 ký số + null) để lưu chuỗi số đọc từ file và buffer_ghi (2000 byte) để lưu trữ kết quả của phổ sao được in ra.

2.2 Code Segment

2.2.1 Bước 1: Lấy dữ liệu đọc được từ tập tin STRING.TXT và kiểm tra tính hợp lệ

Đầu tiên, chương trình thực hiện việc mở và đọc file “STRING.TXT”.

- Mở file: Sử dụng macro `open_file` với chế độ đọc (mode 0). Kết quả trả về (file descriptor) được lưu vào thanh ghi `%v0`. Ta kiểm tra ngay lập tức: nếu `%v0 < 0` (lỗi), chương trình nhảy đến nhãn `error` để in thông báo lỗi.
- Đọc file: Nếu mở thành công, descriptor được lưu vào `%s0`. Chương trình đọc 41 byte từ file vào `buffer_doc`. Số byte thực tế đọc được trả về trong `%v0`, lưu vào `%s1` để dùng làm biến độ dài chuỗi. Sau đó, đóng file ngay để giải phóng tài nguyên.

Kiểm tra dữ liệu: Gọi lệnh `jal kiem_tra` để đảm bảo tính đúng đắn:

- Sử dụng vòng lặp duyệt từng byte trong `buffer_doc`.
- Với mỗi ký tự, so sánh xem nó có nằm trong khoảng mã ASCII của số (48–57) hay không.
- Nếu phát hiện ký tự không hợp lệ, hàm trả về 0; nếu hợp lệ, hàm trả về số lượng chữ số đếm được.
- Quay lại `main`, kiểm tra kết quả: nếu bằng 0 hoặc lớn hơn 40 (quá bộ nhớ đệm cho phép), chương trình nhảy đến `error`.

2.2.2 Bước 2: Chuẩn bị dữ liệu và tìm chiều cao biểu đồ

Sau khi có dữ liệu trong `buffer_doc`, chương trình gọi hàm `jal ve_pho_sao` và thực hiện các bước xử lý như sau:

- **Lưu ngữ cảnh:** Vì hàm này sử dụng nhiều thanh ghi quan trọng (`$s0`, `$s1`, `$s2`), các thanh ghi này cùng với địa chỉ trả về `$ra` được đẩy vào ngăn xếp (stack) để bảo toàn giá trị cũ.
- **Sao chép dòng tiêu đề:** Dòng đầu tiên của file kết quả chính là chuỗi số gốc. Chương trình sử dụng vòng lặp `in_dong1` để chép từng byte từ `buffer_doc` sang `buffer_ghi`, kết thúc bằng ký tự xuống dòng (ASCII 10).
- **Thuật toán tìm Max:** Để vẽ biểu đồ, cần xác định chiều cao tối đa:
 - Khởi tạo `$t1 (max) = 0`.
 - Duyệt toàn bộ chuỗi số: với mỗi ký tự, thực hiện `sub $t4, $t3, 48` để chuyển từ mã ASCII sang giá trị số nguyên.
 - So sánh giá trị này với `$t1`; nếu lớn hơn, cập nhật `$t1`.
 - Kết thúc vòng lặp, giá trị Max được lưu vào `$s2`, dùng làm biến đếm cho vòng lặp vẽ sao tiếp theo.

2.2.3 Bước 3: In ra phổ thỏa mã với yêu cầu

Chương trình sử dụng hai vòng lặp lồng nhau để vẽ biểu đồ từ trên xuống dưới (từ giá trị Max về 1):

- **Vòng lặp ngoài (In hàng):** Biến đếm là `$s2` (chiều cao hiện tại - Level), giảm dần từ Max về 1. Khi `$s2 = 0`, kết thúc việc vẽ.
- **Vòng lặp trong (Duyệt chuỗi số):** Với mỗi hàng (Level), duyệt từng ký tự trong chuỗi đầu vào theo các bước sau:
 - Lấy ký tự tại vị trí đang xét, trừ 48 để lấy giá trị số nguyên (lưu vào `$t3`).
 - So sánh giá trị số này với Level hiện tại (`$s2`):
 - * Nếu giá trị số $\geq$ Level (`bge $t3, $s2`), nhảy đến nhãn `in_sao` để ghi ký tự `*` (ASCII 42) vào bộ đệm.
 - * Ngược lại, ghi ký tự khoảng trắng (ASCII 32) vào bộ đệm.
 - Sau mỗi lần ghi, tăng con trỏ bộ đệm `$t9` lên 1 đơn vị.
- **Xuống dòng:** Sau khi quét hết một hàng (kết thúc vòng lặp trong), ghi ký tự xuống dòng (ASCII 10) vào bộ đệm và giảm Level (`$s2`) để tiếp tục vòng lặp ngoài.

Kết thúc quá trình, `buffer_ghi` sẽ chứa toàn bộ nội dung biểu đồ. Hàm cũng tính toán độ dài chuỗi kết quả và khôi phục các thanh ghi từ stack trước khi thực hiện `jr $ra`.

2.2.4 Bước 4: Kết thúc chương trình

Ghi file: Chương trình mở file `PHO_SAO.TXT` với chế độ 9 (Ghi và Tạo mới). Sử dụng `syscall` để ghi toàn bộ nội dung `buffer_ghi` vào file.

Xuất ra màn hình: Để thuận tiện kiểm tra, chương trình in nội dung `buffer_ghi` ra cửa sổ Console.

Thoát chương trình: Cuối cùng, gọi `syscall 10 (exit)` để trả quyền điều khiển cho hệ điều hành, kết thúc chương trình an toàn.

3 Thống kê số lệnh, loại lệnh (instruction type) sử dụng trong chương trình

3.1 Số lệnh

Có tất cả 5546 lệnh đã thực thi trong chương trình MIPS này, trong đó có 47% lệnh ALU, 12% lệnh jump, 18% lệnh branch, 15% lệnh Memory, và 9% các lệnh khác.

3.2 Các loại lệnh dùng trong chương trình

Lệnh	Loại lệnh
move \$s0, \$v0	R
bltz \$s0, error	I
move \$s1, \$v0	R
la \$a0, buffer_doc	I
move \$a1, \$s1	R
jal kiem_tra	J
move \$s1, \$v0	R
beqz \$s1, error	I
bgt \$s1, 40, error	I
la \$a0, buffer_doc	I
move \$a1, \$s1	R
la \$a2, buffer_ghi	I
jal ve_pho_sao	J
move \$s2, \$v0	R
move \$s3, \$v0	R
bltz \$s3, error	I
li \$t3, 0	I
move \$t0, \$a0	R
move \$t1, \$a1	R
lb \$t2, 0(\$t0)	I
beqz \$t2, kt_done	I
blt \$t2, 48, kt	I
bgt \$t2, 57, kt_fail	I
addi \$t3, \$t3, 1	I
addi \$t0, \$t0, 1	I
addi \$t1, \$t1, -1	I
move \$v0, \$t3	R
jr \$ra	R
li \$v0, 0	I
jr \$ra	R
subi \$sp, \$sp, 16	I
sw \$ra, 12(\$sp)	I
sw \$s0, 8(\$sp)	I
sw \$s1, 4(\$sp)	I
sw \$s2, 0(\$sp)	I
move \$s0, \$a2	R
move \$s1, \$a1	R
move \$t9, \$a2	R

Lệnh	Loại lệnh
move \$t8, \$a0	R
move \$t7, \$s1	R
lb \$t6, 0(\$t8)	I
sb \$t6, 0(\$t9)	I
addi \$t9, \$t9, 1	I
addi \$t8, \$t8, 1	I
addi \$t7, \$t7, -1	I
j in_dong1	J
li \$t6, 10	I
sb \$t6, 0(\$t9)	I
addi \$t9, \$t9, 1	I
move \$t0, \$a0	R
li \$t1, 0	I
move \$t2, \$s1	R
lb \$t3, 0(\$t0)	I
sub \$t4, \$t3, 48	R
bgt \$t4, \$t1, update_max	I
j next_max	J
move \$t1, \$t4	R
addi \$t0, \$t0, 1	I
addi \$t2, \$t2, -1	I
j tim_max	J
move \$s2, \$t1	R
beqz \$s2, chart_done	I
move \$t0, \$a0	R
move \$t1, \$s1	R
beqz \$t1, row_done	I
lb \$t2, 0(\$t0)	I
sub \$t3, \$t2, 48	R
bge \$t3, \$s2, in_sao	I
li \$t4, 32	I
sb \$t4, 0(\$t9)	I
addi \$t9, \$t9, 1	I
j next_col	J
li \$t4, 42	I
sb \$t4, 0(\$t9)	I
addi \$t9, \$t9, 1	I
addi \$t0, \$t0, 1	I
addi \$t1, \$t1, -1	I
j ve_row	J

Lệnh	Loại lệnh
li \$t4, 10	I
sb \$t4, 0(\$t9)	I
addi \$t9, \$t9, 1	I
subi \$s2, \$s2, 1	I
j ve_chart	J
sb \$zero, 0(\$t9)	I
sub \$v0, \$t9, \$s0	R
lw \$s2, 0(\$sp)	I
lw \$s1, 4(\$sp)	I
lw \$s0, 8(\$sp)	I
lw \$ra, 12(\$sp)	I
addi \$sp, \$sp, 16	I
jr \$ra	R

- Lệnh R có tất cả 22 lệnh, trong đó có 3 lệnh thực và 19 lệnh giả.
- Lệnh I có tất cả là 23 lệnh, trong đó có 10 lệnh thực và 13 lệnh giả.
- Lệnh J có tất cả 6 lệnh. Tất cả loại lệnh này là lệnh thực.

4 Tính thời gian chạy của chương trình

Cho thời gian trễ của các khối như sau:

Khối chức năng	Thời gian trễ (ns)
Instruction Memory	150
Registers	100
ALU	100
Data Memory	150
Các khối khác	150

Bảng 2: Thời gian trễ của các khối chức năng

4.1 Xác định chu kỳ CR

Dựa trên bảng độ trễ trên, ta tính chu kỳ CR dựa trên 2 cơ chế:

- Single clock: $T_{CR1} = D_{IM} + D_{Regs} + D_{ALU} + D_{DM} + D_{Regs} = 150 + 100 + 100 + 150 + 100 = 600ns$
- Pipeline: $T_{CR2} = \max(\text{độ trễ từng khối}) = 150ns$

4.2 Xác định IC

4.2.1 Tính IC ở bước 1

Ở bước này, chương trình mở file "STRING.TXT", ta đọc dữ liệu và kiểm tra tính hợp lệ của chương trình. Do không có vòng lặp nên mỗi lệnh chỉ được thực thi một lần duy nhất.

$$IC_{step1} = open_file + move + bltz + read_file + move + close_file + la + move + jal + move + beqz + bgt \\ = 19 \text{ (IC)}$$

4.2.2 Tính IC ở bước 2

Vòng lặp `loop_kt` chạy 40 lần (giả sử đang xét file đầu vào có 40 ký số).

Mỗi lần vòng lặp: chứa 9 lệnh, nên có tất cả $9 * 40 = 360$ (IC).

Kết thúc vòng lặp: có 2 lệnh move,jr.

Lệnh kiểm tra nhánh lỗi: có 2 lệnh li, jr (trường hợp kỹ tự không hợp lệ).

$$IC_{kiem_tra} = 3 + 360 + 2 + 2 = 367$$

4.2.3 Tính IC ở bước 3

Ở bước này, chương trình thực hiện: lưu stack, khởi tạo con trỏ và thanh ghi, in ra các dòng, tìm time max, vẽ chart và kết thúc, tính độ dài.

$$IC_{ve_pho_sao} = IC_{stack} + IC_{khai_tao} + IC_{tim_max} + IC_{ve_chart} + IC_{do_dai} = 5 + 3 + 283 + 284 + 3285 + 8 = 3868$$

4.2.4 Tính IC ở bước 4

Chỉ có 1 lệnh `exit_program`.

4.2.5 Tổng IC của chương trình

$$IC_{total} = IC_{step1} + IC_{kiem_tra} + IC_{ve_pho_sao} + IC_{excit} = 19 + 367 + 3868 + 1 = 4255(IC)$$

4.3 Tính tổng thời gian chạy

- Single Clock : $T_p = 4255 * 600 = 2553000(ns) \approx 2.553(ms)$
- Pipeline (5-stage): $T_p = (42551 + 5) * 150 = 638850(ns) \approx 0.639(ms)$

5 Kết quả kiểm thử

Chương trình sẽ được kiểm thử trên 3 testcase sau để đảm bảo độ chính xác và hiệu quả của thuật toán.

5.1 Test 01

Đây là test mẫu mà đề bài yêu cầu với Input của file là 00100100234567000065432100007898700

Với testcase sau, kết quả khi xuất ra màn hình là :

```
00100100234567000065432100007898700
                                     *
                                   **
                                ***
                             ****
                          *****
                       *******
                    *********
                 **********
             ***********
          *****
       ***
   *
```

```
-- program is finished running --
```


5.2 Test 02

Testcase có chuỗi đầu vào để đọc file là : 12345678902500233553
Với testcase sau, kết quả khi xuất ra màn hình là :

```
12345678902500233553
      *
    * *
  * * *
* * * *
* * * * *
* * * * *
* * * * *
* * * * *
* * * * *
* * * * *
```

5.3 Test 03

Testcase có chuỗi đầu vào để đọc file là : 13579135791357913579135791357913579135
Với testcase sau, kết quả khi xuất ra màn hình là :

```
13579135791357913579135791357913579135
      *      *      *      *      *      *      *
      *      *      *      *      *      *      *
    * *    * *    * *    * *    * *    * *    * *
    * *    * *    * *    * *    * *    * *    * *
  * * *    * * *    * * *    * * *    * * *    * * *
  * * *    * * *    * * *    * * *    * * *    * * *
* * * *    * * * *    * * * *    * * * *    * * * *
* * * *    * * * *    * * * *    * * * *    * * * *
* * * *    * * * *    * * * *    * * * *    * * * *
* * * *    * * * *    * * * *    * * * *    * * * *
```

6 Lời cảm ơn

Nhóm chúng em xin gửi lời cảm ơn chân thành đến Thầy Nguyễn Xuân Minh – giảng viên môn Kiến trúc Máy tính – đã tận tình hướng dẫn, truyền đạt kiến thức và hỗ trợ nhóm trong quá trình thực hiện bài tập lớn.

Những bài giảng và hướng dẫn thực hành của Thầy về MARS và các khái niệm cốt lõi của kiến trúc máy tính đã giúp chúng em tiếp cận bài tập một cách hệ thống và khoa học. Thầy không chỉ là người thầy mà còn là người đồng hành, luôn sẵn sàng lắng nghe và giải đáp mọi thắc mắc của sinh viên.

Chúng em cũng xin ghi nhận những đóng góp quý báu từ Thầy để bài tập ngày một hoàn thiện. Mặc dù đã cố gắng, bài làm không tránh khỏi những hạn chế, và chúng em mong tiếp tục nhận được sự chỉ bảo từ Thầy để tiến bộ hơn trong học tập.

Xin chân thành cảm ơn Thầy!
Kính chúc Thầy sức khỏe và thành công!